Soprillo: Small, but Mechanized SAX

ARMAN AHMED, McGill University, Canada
MAX GROSS, McGill University, Canada
FRÉDÉRIC LAHAIE-BERTRAND, Université de Montréal, Canada

This paper presents a work-in-progress mechanization of the proof theory of the semi-axiomatic sequent calculus (SAX) in the proof assistant Beluga. SAX replaces all noninvertible inference rules with axioms, causing standard cut elimination to fail. Instead, SAX satisfies a modified cut elimination theorem permitting certain analytic cuts called snips to remain, where a snip requires the cut formula to be 'eligible', i.e. a subformula of the principal formula of an axiom. A central challenge in mechanizing SAX is therefore tracking eligibility throughout the proof theory. We address this by representing eligibility as a well-formedness predicate embedded directly within proof judgements rather than imposed through a separate context discipline, adapting a technique due to Crary for encoding substructural constraints as judgements over proof terms. This is analogous to how linearity conditions have been previously handled in mechanizations of session-typed systems. Using this technique, we present mechanized proofs of cut admissibility and the subformula property for a representative propositional fragment, and outline a roadmap for extending the mechanization to the full calculus. To our knowledge, this is the first mechanization of SAX proof theory.

ACM Reference Format:
Arman Ahmed, Max Gross, and Frédéric Lahaie-Bertrand. 2026. Soprillo: Small, but Mechanized SAX. 1, 1 (April 2026), 7 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

The Curry-Howard isomorphism shows the deep relationship between intuitionistic logic and computation. Specifically, it establishes that proofs can be interpreted as programs and proof reduction as computation. Because of this, we can see that the different formalizations of intuitionistic logic will result in different computational interpretations [5]. Howard [8], for example, established the connection between natural deduction and the simply-typed λ-calculus. Curry [3] took a different approach, associating Hilbert-style proofs to combinators. Herbelin [7] established the system LJT based on the sequent calculus with a stoup, leading to a computational interpretation featuring explicit substitutions.

Adding to this lineage, DeYoung et al. [5] presents the semi-axiomatic sequent calculus (SAX), a formalization of intuitionistic logic which arises by replacing all non-invertible rules in the sequent calculus with axioms and leaving the invertible rules unchanged. The resulting system is equivalent to G_3, but standard cut elimination fails: it is replaced by a modified form in which certain analytic cuts, called snips, are permitted to remain in normal-form proofs while preserving the subformula property.

Establishing these results is more involved than in the sequent calculus. Both the admissibility of cut and the subformula property require tracking which formula occurrences are eligible for a snip and which antecedents

Authors' addresses: Arman Ahmed, arman.ahmed@mail.mcgill.ca, McGill University, Montréal, Canada; Max Gross, max.gross@mail.mcgill.ca, McGill University, Montréal, Canada; Frédéric Lahaie-Bertrand, frederic.lahaie-bertrand@umontreal.ca, Université de Montréal, Montréal, Canada.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Association for Computing Machinery.
XXXX-XXXX/2026/4-ART $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

are irrelevant, and both proofs require generalized induction hypotheses to account for how the eligibility of formula occurrences changes through derivations. The original paper necessarily leaves out many portions of the proofs implicit, making SAX an interesting and non-trivial target for mechanization.

As such, we report on a mechanization of the proof theory of Soprillo, a fragment of SAX in Beluga, a proof assistant with its foundations in contextual modal logic and the Logical Framework. Beluga's explicit handling of contexts is particularly suited to mechanizing SAX, precisely because we need to reason about variables in contexts during our proof. Further, since it is an extension of LF, we can build on the existing literature regarding mechanizing similar systems and proofs in the literature [2] [11]. This, to the best of our knowledge, is the first mechanization of SAX and its proof theory. Specifically, we present:

- A mechanization of Soprillo and its translation to and from G_3
- A mechanized proof of the admissibility of cut
- A mechanized proof of cut elimination
- A mechanized proof of the subformula property

The remainder of the paper is organized as follows: Section 2 motivates the mechanization through SAX's computational interpretation and illustrates the proof-theoretic subtleties that make it a challenging target. Section 3 presents our mechanization in detail. Section 4.2 discusses related work and Section 5.1 concludes.

2 MOTIVATION

2.1 Computational Interpretation of SAX

Beyond its proof-theoretic results, SAX admits a natural operational interpretation as a system of concurrent processes that interact asynchronously. This interpretation has been studied across several computational models, from shared memory concurrency [5] to message-passing concurrency [], and each of these interpretations has been shown to satisfy desirable properties such as progress, preservation, confluence, and termination. Specifically, under the shared memory interpretation, variables are substituted by addresses at runtime, and thus the sequent

$$x_1 : A_1, \ldots, x_n : A_n \vdash P :: (z : C)$$

should be understood as typing a process P that reads from addresses $x_1, \ldots, x_n$ and writes to address z. The operational behaviour of processes is then determined by the rules of the proof system, for instance cut and snip allocates a fresh memory cell, spawning a writer process and a reader process that synchronize through it, and process expressions assigned to right rules write to memory while those assigned to left rules read from it.

This low-level interpretation of SAX further gives rise, in extensions of SAX, to type-theoretically justified compiler optimizations such as in-place memory reuse []. Extensions of SAX in adjoint logic have also served as the logical foundation for concurrent language implementations []. All of these applications ultimately depend on the soundness of SAX's proof theory.

2.2 Mechanization

3 CONTRIBUTIONS

3.1 Core Proof-Theoretic Results

Listed below are the proof-theoretic results that we have mechanized for our representative subset of SAX, covering identity (`id`), implication ($\supset R$, $\supset L^0$), tensor ($\otimes R^0$, $\otimes L$), and snips ($\texttt{snip}^1$, $\texttt{snip}^2$).

Theorem 1 (Translations between G_3 and SAX). *$\Gamma \vdash A$ in G_3 iff $\Gamma \vdash A$ in SAX.*

In the following theorems, we say a derivation is cut-free if it contains no uses of cut, but may contain snips.

Theorem 2 (Admissibility of Cut). *If there are cut-free derivations of* $\Gamma \vdash x : A$ *and* $\Gamma, x : A \vdash z : C$*, then there exists a cut-free derivation of* $\Gamma \vdash z : C$*.*

Theorem 3 (Cut Elimination in SAX). *If there is a derivation of* $\Gamma \vdash x : A$*, then there is a cut-free derivation of the same sequent.*

Theorem 4 (Subformula Property). *All formulas in a cut-free derivation of* $\Gamma \vdash x : A$ *are subformulas of formulas in* Γ *or* A*.*

3.2 Encoding SAX in Beluga

3.3 Eligibility and Irrelevance

```
LF lconcS : o → type =
  ...
  | l⊃RS : (lhypS A → lconcS B) → lconcS (A ⊃ B)
  | l⊃L0S : lhypS A → lhypS (A ⊃ B) → lconcS B
  ...
and
LF elig_c : lconcS A → type =
  | elig_c/l⊃L0S : {x : lhypS A} {y : lhypS (A ⊃ B)}
                   elig_c (l⊃L0S x y)
and
LF elig_h : (lhypS A → lconcS B) → type =
  | elig_h/l⊃L0S : {y : lhypS (A ⊃ B)}
                   elig_h (\x. l⊃L0S x y)
;
```

3.4 Translation Between G3 and SAX

Theorem 1 states that G_3 and SAX derive the same sequents. We mechanize both directions as recursive functions over derivations that translate hypotheses between the two systems using a relation pairing up the two contexts hypothesis by hypothesis. Because SAX inherits the invertible rules from G_3, they are translated in both directions by directly applying the corresponding rule. The interesting cases are the noninvertible rules, which correspond to axioms in SAX.

The direction from G_3 to SAX, showing that G_3 derives no more than SAX, proceeds by showing that each noninvertible G_3 rules can be derived using cuts and axioms. For instance, $\supset L$ which has no direct counterpart in SAX, can be simulated using two nested cuts against the axiom $\supset L^0$:

```
rec seq2sax : (A : [⊢ o]) rel-ctx-S [Γ] [Γ'] → [Γ ⊢ conc A[]] → [Γ' ⊢ concS A[]] =
/ total 2 /
fn R D ⇒ case D of
  | [Γ ⊢ ⊃L c1 (\h. c2) H] ⇒
    let [Γ' ⊢ HS] = hyp2hypS R [Γ ⊢ H] in
    let [Γ' ⊢ c1S] = seq2sax R [Γ ⊢ c1] in
    let [Γ', x : hypS B[] ⊢ c2S] = seq2sax (cons-S R) [Γ, x : hyp _ ⊢ c2] in
    [Γ' ⊢ cutS (cutS c1S (\h. ⊃L0S h HS[..])) (\h. c2S)]
  ...
;
```

The other direction, from SAX to G_3, shows that each SAX axiom can be derived in G_3 using the corresponding rule and identity. For instance, $\supset L^0$ is translated as an application of $\supset L$ in G_3 with identity proofs for both premises:

```
rec sax2seq : (A : [⊢ o]) rel-S-ctx [Γ'] [Γ] → [Γ' ⊢ concS A[]] → [Γ ⊢ conc A[]] =
/ total 2 /
fn R D ⇒ case D of
  | [Γ' ⊢ ⊃L0S HS1 HS2] ⇒
    let [Γ ⊢ H1] = hypS2hyp R [Γ' ⊢ HS1] in
    let [Γ ⊢ H2] = hypS2hyp R [Γ' ⊢ HS2] in
    [Γ ⊢ ⊃L (initial H1) (\h. initial h) H2]
  ...
;
```

3.5 Admissibility of Cut and Cut Elimination

```
rec cut-admis : {Γ : lctxS} {A : [⊢ o]} [Γ ⊢ lconcS A[]]
                → [Γ, x:lhypS A[] ⊢ lconcS C[]]
                → [Γ ⊢ lconcS C[]] =
/ total 1 /
mlam Γ A ⇒ fn D E ⇒
case (D, E) of
  %% */_
  | ([Γ ⊢ l⊃L0S h1 h2], [Γ, x : lhypS _ ⊢ E]) ⇒
    [Γ ⊢ lsnip1S (l⊃L0S h1 h2) (elig_c/l⊃L0S h1 h2) (\h. E[..,h])]
  %% ⊃R/⊃L0
  | ([Γ ⊢ l⊃RS (\h. D1)], [Γ, x : lhypS _ ⊢ l⊃L0S h1 h2]) ⇒
    (case ([Γ, x : lhypS _ ⊢ h2], [Γ, x : lhypS _ ⊢ h1]) of
      | ([Γ, x : lhypS _ ⊢ x], [Γ, x : lhypS _ ⊢ #h1[..] ]) ⇒
        [Γ ⊢ D1[..,#h1]]
      | ([Γ, x : lhypS _ ⊢ #h2[..]], [Γ, x : lhypS _ ⊢ x]) ⇒
        [Γ ⊢ lsnip2S (l⊃RS (\h. D1)) (\h. l⊃L0S h #h2[..]) (elig_h/l⊃L0S #h2[..])]
      | ([Γ, x : lhypS _ ⊢ #h2[..]], [Γ, x : lhypS _ ⊢ #h1[..]]) ⇒
        [Γ ⊢ l⊃L0S #h1 #h2]
    )
  %% _/Snip1
  | ([Γ ⊢ D], [Γ, x : lhypS _ ⊢ lsnip1S E1 e (\h. E2)]) ⇒
    let [Γ ⊢ F1] = cut-admis _ _ [Γ ⊢ D] [Γ, x : lhypS _  ⊢ E1] in
    let [Γ, x : lhypS _ ⊢ F2] =
      cut-admis _ _ [Γ, y:lhypS _ ⊢ D[..]] [Γ, x : lhypS _, y : lhypS _ ⊢ E2[..,x]] in
    cut-admis _ _ [Γ ⊢ F1] [Γ, y : lhypS _ ⊢ F2]
  %% _/Snip2
  | ([Γ ⊢ D], [Γ, x : lhypS _ ⊢ lsnip2S E1 (\h. E2) e]) ⇒
    let [Γ ⊢ F1] = cut-admis _ _ [Γ ⊢ D] [Γ, x : lhypS _  ⊢ E1] in
    let [Γ,  x : lhypS _ ⊢ F2] =
      cut-admis _ _ [Γ, y:lhypS _ ⊢ D[..]] [Γ, x : lhypS _, y : lhypS _ ⊢ E2[..,x]] in
    cut-admis _ _ [Γ ⊢ F1] [Γ, y : lhypS _ ⊢ F2]
```

```
  ...
  ;

rec cut-elim : {Γ : lctxS} [Γ ⊢ lconcS-cut A[]] → [Γ ⊢ lconcS A[]] = ...
```

3.6 Subformula Property

```
inductive subfProp : {Γ : lctxS} {Σ : [⊢ formlist]} {D : [Γ ⊢ lconcS A[]]} ctype =
  | subfProp/lsnip1 :
      (D : [Γ ⊢ lconcS A[]])
      (E: [Γ, x : lhypS A[] ⊢ lconcS B[]])
      {e : [Γ ⊢ elig_c D]}
      subfProp [Γ] [⊢ Σ] [Γ ⊢ D]
      → [⊢ inSig A Σ]
      → subfProp [Γ, x : lhypS A[]] [⊢ Σ] [Γ, x : lhypS A[] ⊢ E]
      → subfProp [Γ] [⊢ Σ] [Γ ⊢ lsnip1S D e (\x. E[.., x])]
  | subfProp/lsnip2 :
      {e : [Γ  ⊢ elig_h (\x. E)]}
      subfProp [Γ] [⊢ Σ] [Γ ⊢ D]
      → [⊢ inSig A Σ]
      → subfProp [Γ, x : lhypS A[]] [⊢ Σ] [Γ, x : lhypS A[] ⊢ E]
      → subfProp [Γ] [⊢ Σ] [Γ ⊢ lsnip2S D (\x. E[.., x]) e]
  ...
  ;

rec subformula-property : {Γ : lctxS} {D : [Γ ⊢ lconcS A[]]}
                          → ctxInSig  [Γ] [⊢ Σ]
                          → [⊢ inSig A Σ]
                          → subfProp [Γ] [⊢ Σ] [Γ ⊢ D] =
/ total 1 /
mlam Γ D ⇒ fn ci goal ⇒
case [Γ ⊢ D] of
  | [Γ ⊢ lsnip1S D' e (\h. E')] ⇒
    (case [Γ ⊢ e] of
      | [Γ ⊢ elig_c/l⊃L0S #h1 #h2] ⇒
        let ih2    = lookup ci [Γ ⊢ #h2] in
        let isig_B = subf-inSig [⊢ subf/⊃2 subf/refl] ih2 in
        let ih1    = lookup ci [Γ ⊢ #h1] in
        let sp_D   = subfProp/l⊃L0S [Γ ⊢ #h1] [Γ ⊢ #h2] ih1 ih2 in
        let ci'    = ctxInSig/cons ci isig_B in
        let sp_E   = subformula-property _ [Γ, h : lhypS _ ⊢ E'] ci' goal in
        subfProp/lsnip1 _ sp_D isig_B sp_E
    )
  | [Γ ⊢ lsnip2S D' (\h. E') e] ⇒
    (case [Γ ⊢ e] of
      | [Γ ⊢ elig_h/l⊃L0S #y] ⇒
```

```
        let ih_y   = lookup ci [Γ ⊢ #y] in
        let isig_A = subf-inSig [⊢ subf/⊃1 subf/refl] ih_y in
        let ci'    = ctxInSig/cons ci isig_A in
        let sp_D   = subformula-property _ [Γ ⊢ D'] ci isig_A in
        let sp_E   = subformula-property _ [Γ, h : lhypS _ ⊢ E'] ci' goal in
        subfProp/lsnip2 _ sp_D isig_A sp_E
    )
  ...
  ;
```

4 RELATED WORK

4.1 SAX and Related Systems

Francalaza et. al. [6], present an alternate formulation of SAX embedded in adjoint logic. The resulting computational interpretation models message-passing concurrency: each variable in their system becomes a channel, whose properties are modeled by its adjoint modality, and each consequent becomes a provider of said channels [6]. They present an implementation of this language in Go [6]. Logically then, Pfenning and Pruiksma [12] present SAX with recursion as a unifying framework between message-passing and shared memory. Specifically, they show that the shared-memory semantics and the message-passing semantics have a strong bisimulation on well-typed terms [12].

DeYoung and Pfenning [4] present SNAX, a modification of the standard type theory of SAX that accounts for data layout in shared memory concurrency. It achieves this, in part, by making eligibility a first-class part of the system, rather than a label, as described above; they also add recursion to the system [4]. Ng and Pfenning [10] present SNAX with reuse, a single-threaded language based on SNAX that models memory reuse.

Boyland and Pfenning [1] present an additional adjoint-logical formulation of SAX with bidirectional typing rules. They further show a compilation from adjoint natural deduction [9] to their adjoint system, building the groundwork for a compiler of adjoint-functional programs [1]. Somayyajula and Pfenning [14] applies a type refinement system to SAX, allowing them to reason about the partial correctness of SAX programs. Their framework also allows for the synthesis of SAX with Hoare logic [14].

4.2 Related Mechanizations

Pfenning [11] presents an elegant proof for cut-elimination in the sequent calculus with the same structure as in [5], and our mechanization. He also presents a formalization of this proof in LF [11]. Crary [2] describes a method for encoding linearity in LF without any linear or modal extensions. We used his techniques to in our representation of SAX in Beluga to properly represent eligible formulas in our system. These act similarly to propositions in linear logic as every eligible formula must be used in a SAX proof. We also drew on the techniques described in [13] to properly represent SAX in Beluga.

5 CONCLUSION

We report on a mechanization of SAX and a mechanization of the proof of its translation to the sequent calculus. We also mechanize the proofs of cut-elimination, cut-admissibility, and the subformula property for Soprillo, a fragment of SAX containing only snips, implication, and the identity rule.

5.1 Future Work

The most obvious avenue for future work is to extend our mechanizations to fragments larger than Soprillo. While extending these proofs to the complete SAX system may be infeasible, mechanizations that include ⊗

would be of great use in showing the adaptability of our representations and proof techniques used. In a similar vein, a mechanization of SAX within adjoint logic would allow for similar meta-theoretic guarantees regarding the message-passing interpretation of SAX. Further, a mechanization of SNAX should follow relatively easily from our mechanization.

Another natural step is a mechanization of the shared memory interpretation of SAX, along with its properties including progress and termination. [5] presents a logical relations argument for the latter, which are particularly amenable to mechanization. We suspect that the dynamic semantics of the shared memory interpretation, which takes the form of multiset rewriting rules on configurations (allocated memory cells) can be represented as ordered lists in Beluga. While we intended to explore this path further, our initial challenges at properly representing eligibility in Beluga stalled the project significantly.

REFERENCES

[1] Joanna Boyland and Frank Pfenning. 2025. Compiling adjoint natural deduction to the semi-axiomatic sequent calculus. Abstract, 9th International Workshop on Trends in Linear Logic and its Applications (TLLA 2025), affiliated with FSCD 2025, Birmingham, UK, 19–20 July 2025. https://lipn.univ-paris13.fr/TLLA/2025/abstracts/06-boyland-pfenning.pdf

[2] Karl Crary. 2010. Higher-order representation of substructural logics. In *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming* (Baltimore, Maryland, USA) *(ICFP '10)*. Association for Computing Machinery, New York, NY, USA, 131–142. https://doi.org/10.1145/1863543.1863565

[3] H. B. Curry. 1934. Functionality in Combinatory Logic*. *Proceedings of the National Academy of Sciences* 20, 11 (1934), 584–590. https://doi.org/10.1073/pnas.20.11.584 arXiv:https://www.pnas.org/doi/pdf/10.1073/pnas.20.11.584

[4] Henry DeYoung and Frank Pfenning. 2023. Data Layout from a Type-Theoretic Perspective. *Electronic Notes in Theoretical Informatics and Computer Science* 1 (feb 2023). https://doi.org/10.46298/entics.10507 Proceedings of MFPS XXXVIII.

[5] Henry DeYoung, Frank Pfenning, and Klaas Pruiksma. 2020. Semi-Axiomatic Sequent Calculus. In *5th International Conference on Formal Structures for Computation and Deduction (FSCD 2020) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 167)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 29:1–29:22. https://doi.org/10.4230/LIPIcs.FSCD.2020.29

[6] Adrian Francalanza, Gerard Tabone, and Frank Pfenning. 2024. Implementing a Message-Passing Interpretation of the Semi-Axiomatic Sequent Calculus (Sax). In *Coordination Models and Languages - 26th IFIP WG 6.1 International Conference, COORDINATION 2024, Held as Part of the 19th International Federated Conference on Distributed Computing Techniques, DisCoTec 2024, Groningen, The Netherlands, June 17-21, 2024, Proceedings (Lecture Notes in Computer Science, Vol. 14676)*. Springer, 295–313. https://doi.org/10.1007/978-3-031-62697-5_16

[7] Hugo Herbelin. 1995. A Lambda-Calculus Structure Isomorphic to Gentzen-Style Sequent Calculus Structure. In *Computer Science Logic (CSL '94) (Lecture Notes in Computer Science, Vol. 933)*, L. Pacholski and J. Tiuryn (Eds.). Springer, Kazimierz, Poland, 61–75.

[8] W. A. Howard. 1980. The Formulae-as-Types Notion of Construction. In *To H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*. Academic Press, 479–490. Original unpublished note from 1969.

[9] Junyoung Jang, Sophia Roshal, Frank Pfenning, and Brigitte Pientka. 2024. Adjoint Natural Deduction (Extended Version). arXiv:2402.01428 [cs.LO] https://arxiv.org/abs/2402.01428 Submitted on 2 Feb 2024.

[10] Daniel Ng. 2024. Memory Reuse in Linear Functional Computation. https://www.cs.cmu.edu/~fp/courses/15417-s25/misc/Ng24.pdf Accessed: 2026-04-21.

[11] Frank Pfenning. 2000. Structural Cut Elimination: I. Intuitionistic and Classical Logic. *Information and Computation* 157, 1–2 (2000), 84–141. https://doi.org/10.1006/inco.1999.2832

[12] Frank Pfenning and Klaas Pruiksma. 2023. Relating Message Passing and Shared Memory, Proof-Theoretically. In *Coordination Models and Languages*, Sung-Shik Jongmans and Antónia Lopes (Eds.). Springer Nature Switzerland, Cham, 3–27.

[13] Chuta Sano, Ryan Kavanagh, and Brigitte Pientka. 2023. Mechanizing Session-Types using a Structural View: Enforcing Linearity without Linearity. *Proc. ACM Program. Lang.* 7, OOPSLA2, Article 235 (Oct. 2023), 26 pages. https://doi.org/10.1145/3622810

[14] Siva Somayyajula and Frank Pfenning. 2023. Dependent Type Refinements for Futures. *Electronic Notes in Theoretical Informatics and Computer Science* Volume 3-Proceedings of... (Nov. 2023). https://doi.org/10.46298/entics.12286

[15] Tesla Zhang, Sonya Simkin, Rui Li, Yue Yao, and Stephanie Balzer. 2025. A Language-Agnostic Logical Relation for Message-Passing Protocols. arXiv:2506.10026 [cs.PL] https://arxiv.org/abs/2506.10026